

Setup Guide

SaaS Web Application on DigitalOcean Kubernetes (DOKS)

Customer Success Engineering — written to train the customer's engineering team to independently deploy, operate, and troubleshoot this environment.

Who this is for and what you'll end up with

This guide assumes no prior DigitalOcean Kubernetes experience. By the end, your team will be able to independently:

- Push a container image to a private DigitalOcean Container Registry (DOCR).
- Create and connect to a DigitalOcean Kubernetes (DOKS) cluster.
- Deploy the app with a public Load Balancer and automatic scaling.
- Verify, update, roll back, and tear down the environment.

Every DigitalOcean-specific step links to the official documentation page it's based on, so your team has a durable reference beyond this document.

Prerequisites

- A DigitalOcean account with billing enabled.
- A DigitalOcean API token with read/write scope (create one from the control panel under API → Generate New Token).
- Docker installed locally, OR a browser-based environment such as GitHub Codespaces if local installation isn't possible.
- Your application code with a working Dockerfile (this guide uses a Flask app listening on port 5000 as the example).

📖 *DigitalOcean docs:* [How To Generate a Personal Access Token](#)

Step 1 — Install and authenticate doctl

doctl is DigitalOcean's official command-line tool. It's how you'll create and manage the registry and cluster from a terminal.

1. Install doctl for your operating system (macOS: Homebrew; Linux: Snap or direct binary; Windows: direct binary). If you're in a browser-only environment like GitHub Codespaces, download the Linux binary directly:

```
cd /tmp
wget https://github.com/digitalocean/doctl/releases/download/v1.104.0/doctl-1.104.0-linux-amd64.tar.gz
tar xf doctl-1.104.0-linux-amd64.tar.gz
sudo mv doctl /usr/local/bin
doctl version
```

2. Authenticate doctl with your DigitalOcean account:

```
doctl auth init
```

Paste your API token when prompted. A confirmation message means it worked.

📖 *DigitalOcean docs:* [How to Install and Configure doctl](#)

Step 2 — Create a Container Registry and push your image

1. Create a private registry (skip if one already exists):

```
doctl registry create <your-registry-name> --subscription-tier basic
```

2. Log Docker in to the registry:

```
doctl registry login
```

3. Build your image, tagging it with the full registry path and a version:

```
docker build -t registry.digitalocean.com/<your-registry-name>/<image-name>:v1 .
```

4. Push it:

```
docker push registry.digitalocean.com/<your-registry-name>/<image-name>:v1
```

📌 Tag with a real version (v1, v2, ...) rather than always pushing "latest". Kubernetes deployments reference an exact tag, which is what makes rollbacks possible later.

📖 *DigitalOcean docs:* [Container Registry Quickstart](#)

📖 *DigitalOcean docs:* [How to Use Your Private Registry with Docker and Kubernetes](#)

Step 3 — Create the DOKS cluster

You can do this from the control panel (Create → Kubernetes) or the CLI. The CLI is faster to repeat and document:

```
doctl kubernetes cluster create <cluster-name> \  
  --region <region-slug> \  
  --node-pool "name=default-pool;size=s-2vcpu-4gb;count=2;auto-scale=true;min-nodes=2;max-nodes=4"
```

Choose a region close to your users, and match your Container Registry's region if possible to reduce image-pull latency and avoid cross-region transfer. This takes 3–5 minutes to provision.

📌 `auto-scale=true` with `min/max nodes` lets the underlying node pool grow if pod-level autoscaling (Step 6) ever needs more room to schedule pods — without this, pods can get stuck Pending even though the HPA says to scale up.

📖 *DigitalOcean docs:* [How to Create Kubernetes Clusters](#)

📖 *DigitalOcean docs:* [Kubernetes Quickstart](#)

Step 4 — Connect kubectl to the cluster

```
doctl kubernetes cluster kubeconfig save <cluster-name>  
kubectl get nodes
```

The first command downloads cluster credentials and points kubectl's active context at your cluster. The second confirms the connection — you should see your node(s) with STATUS: Ready.

📖 *DigitalOcean docs:* [How to Connect to a DigitalOcean Kubernetes Cluster](#)

Step 5 — Link the registry to the cluster

```
doctl kubernetes cluster registry add <cluster-name>
```

This authorizes the cluster's nodes to pull images from your private registry automatically. Without it, pods fail to start with an ImagePullBackOff error.

📖 *DigitalOcean docs:* [How to Use Your Private Registry with Docker and Kubernetes](#)

Step 5.5 — Install metrics-server (required for autoscaling)

📌 Unlike some managed Kubernetes offerings, DOKS does NOT install metrics-server by default. Skip this step and your HorizontalPodAutoscaler will sit at TARGETS: <unknown>/60% forever and never scale — this trips up almost everyone the first time.

1. Install the upstream metrics-server:

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
```

2. Wait about a minute, then confirm metrics are flowing:

```
kubectl top nodes
```

Once this returns real CPU/memory numbers instead of an error, metrics-server is working and the HPA (Step 6.4) will be able to read live metrics.

Using 1-Click Marketplace

1. Go to the Kubernetes Metrics Server Marketplace Page.
2. Click the **Deploy to DigitalOcean** button.
3. Select your destination cluster to automatically install

🔗 DigitalOcean docs: [Autoscale a Cluster With Horizontal Pod Autoscaling](#)

🔗 DigitalOcean docs: [Kubernetes Metrics Server \(Marketplace 1-click alternative\)](#)

Step 6 — Deploy the application

Four manifest files describe the desired state of the app. Create a k8s/ folder and add each file below, then apply them.

6.1 Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: saas-app
```

Keeps this app's resources logically separated from anything else that later runs on the cluster.

6.2 Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: saas-web-app
  namespace: saas-app
spec:
  replicas: 2
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: saas-web-app
  template:
    metadata:
      labels:
        app: saas-web-app
    spec:
      containers:
        - name: saas-web-app
          image: registry.digitalocean.com/<your-registry-name>/<image-name>:v1
          ports:
            - containerPort: 5000
          resources:
            requests:
```

```

      cpu: "250m"
      memory: "128Mi"
    limits:
      cpu: "500m"
      memory: "256Mi"
    readinessProbe:
      httpGet:
        path: /healthz
        port: 5000
    livenessProbe:
      httpGet:
        path: /healthz
        port: 5000

```

Runs 2 copies of the app. maxUnavailable: 0 guarantees zero-downtime rolling updates. The readiness/liveness probes hit a lightweight /healthz endpoint so Kubernetes knows when a pod can safely receive traffic and when one needs to be restarted.

6.3 Service (Load Balancer)

```

apiVersion: v1
kind: Service
metadata:
  name: saas-web-app
  namespace: saas-app
  annotations:
    service.beta.kubernetes.io/do-loadbalancer-name: "saas-web-app-lb"
    service.beta.kubernetes.io/do-loadbalancer-healthcheck-path: "/healthz"
    service.beta.kubernetes.io/do-loadbalancer-healthcheck-port: "5000"
spec:
  type: LoadBalancer
  selector:
    app: saas-web-app
  ports:
    - port: 80
      targetPort: 5000

```

type: LoadBalancer tells DigitalOcean's cloud controller manager to automatically provision a real, billed DO Load Balancer and give it a public IP that routes traffic to healthy pods.

🔗 [DigitalOcean docs: How to Add Load Balancers to Kubernetes Clusters](#)

🔗 [DigitalOcean docs: How to Configure Advanced Load Balancer Settings in Kubernetes Clusters](#)

6.4 HorizontalPodAutoscaler

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: saas-web-app-hpa
  namespace: saas-app
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: saas-web-app
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource

```

```
resource:
  name: cpu
  target:
    type: Utilization
    averageUtilization: 60
```

Automatically adds pods when average CPU usage exceeds 60% of what each pod requested, and removes them as load drops — between a floor of 2 and a ceiling of 6. This depends on metrics-server, which you installed in Step 5.5 — without it, this section shows TARGETS: <unknown> and never scales.

6.5 Apply everything

```
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/
kubectl -n saas-app rollout status deployment/saas-web-app
```

The namespace is applied first and separately because the other three files declare namespace: saas-app, which must already exist.

Step 7 — Verify the deployment

```
kubectl -n saas-app get pods
kubectl -n saas-app get svc saas-web-app
```

Wait for pods to show 1/1 Running, and for the Service's EXTERNAL-IP column to populate (usually 60–90 seconds — DigitalOcean is provisioning the actual Load Balancer resource behind the scenes). Then test it:

```
curl http://<EXTERNAL-IP>/
curl http://<EXTERNAL-IP>/healthz
```

You should get your app's homepage back, and {"status": "healthy"} from the health endpoint.

Step 8 — Prove autoscaling works

Open two terminals. In the first, watch the autoscaler live:

```
kubectl -n saas-app get hpa saas-web-app-hpa -w
```

In the second, generate CPU load against the app (adjust the URL path to whatever load-generating endpoint your app exposes):

```
kubectl -n saas-app run load-generator --rm -i --tty --restart=Never \
  --image=busybox -- /bin/sh -c \
  "while true; do wget -q -O- http://saas-web-app.saas-
  app.svc.cluster.local/load; done"
```

Within 1–2 minutes you should see REPLICAS climb in the first terminal. Stop the load generator (Ctrl+C) and replicas will scale back down after about 5 minutes.

Day-2 operations

Ship a new version (zero downtime)

```
docker build -t registry.digitalocean.com/<your-registry-name>/<image-name>:v2 .
docker push registry.digitalocean.com/<your-registry-name>/<image-name>:v2
kubectl -n saas-app set image deployment/saas-web-app \
  saas-web-app=registry.digitalocean.com/<your-registry-name>/<image-name>:v2
kubectl -n saas-app rollout status deployment/saas-web-app
```

Roll back a bad release

```
kubectl -n saas-app rollout undo deployment/saas-web-app
```

Check logs

```
kubectl -n saas-app logs -l app=saas-web-app --tail=100 -f
```

Tear down (avoid ongoing charges)

```
kubectl delete -f k8s/
doctl kubernetes cluster delete <cluster-name>
doctl registry delete <your-registry-name>
```

⚠ Delete the Service (LoadBalancer) before deleting the cluster. This deprovisions the actual DigitalOcean Load Balancer resource so you stop being billed for it.

Troubleshooting

Symptom	Likely cause	Fix
EXTERNAL-IP stuck on <pending>	Load Balancer still provisioning, or account resource limit reached	Wait 2–3 minutes; check Networking → Load Balancers in the DO console for errors
ImagePullBackOff	Registry not linked to cluster, or wrong image path/tag	Re-run doctl kubernetes cluster registry add; confirm the image: field matches exactly what you pushed
Pods CrashLoopBackOff	App error on startup, or wrong port/health path	kubectl logs <pod-name> -n saas-app for the stack trace

Reference documentation

Every link used in this guide, in one place, for your team's bookmarks:

Topic	Link
Generate a Personal Access Token	https://docs.digitalocean.com/reference/api/create-personal-access-token/
Install and configure doctl	https://docs.digitalocean.com/reference/doctl/how-to/install/
Container Registry quickstart	https://docs.digitalocean.com/products/container-registry/getting-started/quickstart/
Use your private registry with Docker and Kubernetes	https://docs.digitalocean.com/products/container-registry/how-to/use-registry-docker-kubernetes/
Enable push-to-deploy with GitHub Actions	https://docs.digitalocean.com/products/container-registry/how-to/enable-push-to-deploy/
Kubernetes quickstart	https://docs.digitalocean.com/products/kubernetes/getting-started/quickstart/
How to create Kubernetes clusters	https://docs.digitalocean.com/products/kubernetes/how-to/create-clusters/
How to connect to a Kubernetes cluster	https://docs.digitalocean.com/products/kubernetes/how-to/connect-to-cluster/
Autoscale a cluster with HPA (metrics-server setup)	https://docs.digitalocean.com/products/kubernetes/how-to/set-up-autoscaling/
Kubernetes Metrics Server (Marketplace)	https://docs.digitalocean.com/products/marketplace/catalog/kubernetes-metrics-server/
How to add Load Balancers to Kubernetes clusters	https://docs.digitalocean.com/products/kubernetes/how-to/add-load-balancers/
Advanced Load Balancer settings in Kubernetes	https://docs.digitalocean.com/products/kubernetes/how-to/configure-load-balancers/
All Kubernetes how-to guides	https://docs.digitalocean.com/products/kubernetes/how-to/